

Vector databases and libraries for RAG

Comparison of FAISS, Milvus, NanoVectorDB-style local stores, MongoDB, OpenSearch, PostgreSQL (pgvector), Chroma, and Qdrant. Oriented toward semantic search, ANN, metadata filters, and operations.

Quick positioning

System	What it is	Typical deployment	Best for
FAISS	Library (in-process ANN), not a DB	Embedded / batch	Max throughput on one machine; you build persistence and APIs
Milvus	Purpose-built vector DB	Cluster or managed (e.g. Zilliz)	Large scale, filtering, hybrid patterns
NanoVectorDB / local	Tiny embedded store (file + simple ANN)	Same process as app	Demos, single node, zero ops
MongoDB Atlas	Document DB + vector search	Managed Atlas	Teams on Mongo; unify docs + vectors
OpenSearch	Search engine + k-NN	Self-hosted or AWS	Logs + text + vectors; BM25 + dense
PostgreSQL + pgvector	Relational DB + vector extension	RDS, Cloud SQL, self-hosted	Vectors with transactional data, SQL, ACID
Chroma	Developer-centric embedded/client DB	Local or server	Fast prototypes, Python-centric RAG
Qdrant	Purpose-built vector DB	Self-hosted or Qdrant Cloud	Solid API, filtering; middle ground scale

Tradeoffs that matter for RAG

1. Is it a database?

FAISS is an index library: no built-in multi-tenant auth, replication, backup, or query server unless you wrap it. The others expose a server API or live inside a full DB.

2. Scale and operations

Rough tendency: FAISS (DIY) and nano-style local stores minimize operational surface but you own everything. Chroma is easy. Qdrant and Milvus suit serious vector workloads and clustering. OpenSearch is heavier operationally but strong when search is central. Postgres offers the best data-model integration; vector scale needs tuning. Mongo Atlas is easy if you are already on Mongo.

3. Hybrid search

OpenSearch / Elasticsearch: strongest story for BM25 + dense in one stack. Milvus and Qdrant: dense + metadata filter is first-class; full-text hybrid often means another component or extensions. Postgres: SQL + tsvector + pgvector can hybrid well in one engine for many apps.

4. Filtering and consistency

Postgres and Mongo shine when retrieval must respect row-level business rules and transactions. Pure vector stores excel at ANN; you still design source of truth unless the whole app lives there.

5. Latency and embedding churn

High insert/update rates (frequent re-embedding) favor engines built for streaming upserts (Milvus, Qdrant, managed vector tiers). FAISS often favors batch rebuild unless you use incremental index types carefully.

Practical selection heuristics

- Prototype / laptop / CI: Chroma, nano-style local, or SQLite/pgvector in Docker.
- Production where vectors are the product: Qdrant or Milvus (or managed equivalents).
- Vectors as a column next to orders/users: Postgres + pgvector (or Mongo if that is your primary store).
- Logs + dashboards + search + vectors: OpenSearch.
- ANN only in Python on one big machine: FAISS.
- Avoid two data systems: sometimes Postgres, Mongo, or OpenSearch alone is enough if vector features meet SLOs.

Generated for RAG-Anything / LightRAG context. Regenerate with: python scripts/generate_vector_db_comparison_pdf.py